

Familiar template syntax for generic lambdas

Document #: P0428R2
Date: 2017-07-13
Project: Programming Language C++
Audience: Core Working Group
Reply-to: Louis Dionne <ldionne.2@gmail.com>

1 Revision history

- R0 – Initial draft
- R1 – Incorporate feedback from CWG in Kona:
 - Rebase on top of the C++17 DIS
 - Add alternative term in the grammar for lambda expressions instead of using *opt* template parameter list
 - Adjust wording for the template parameter list of the conversion-to-function-pointer operator
 - Make sure that a lambda with a template parameter list is a generic lambda
 - Shorten discussion
- R2 – Move the definition of a generic lambda from **dcl.spec.auto** to **expr.prim.lambda** per CWG instructions in Toronto.

2 Introduction

C++14 added the ability to define generic lambdas, i.e. lambdas where the **operator()** of the generated *closure-type* is a template. This addition was initially proposed in [N3418], which included many different features for generic lambdas, including the functionality proposed by this paper. However, N3418 was not accepted as-is and its successor, [N3559], was accepted instead. N3559 settled on the **auto**-based syntax that we know in C++14 for defining generic lambdas, leaving the usual template syntax out for lack of clear use cases (according to an author of N3559):

```
[](auto x) { /* ... */ }
```

Unfortunately, this syntax makes it difficult to interact with the type of the parameter(s) and lacks flexibility that is sometimes required, as outlined in the [Motivation](#) section. Hence, this paper proposes adding the ability to use the familiar template syntax when defining lambda expressions:

```

[]<typename T>(T x) { /* ... */ }
[]<typename T>(T* p) { /* ... */ }
[]<typename T, int N>(T (&a)[N]) { /* ... */ }

```

3 Motivation

There are a few key reasons why the current syntax for defining generic lambdas is deemed insufficient by the author. The gist of it is that some things that can be done easily with normal function templates require significant hoop jumping to be done with generic lambdas, or can't be done at all. The author thinks that lambdas are valuable enough that C++ should support them just as well as normal function templates. The following details such areas where lambdas are lacking in their current form:

1. The limited form of "pattern matching" on template argument allowed by C++ in function templates is very useful, and it would be equally useful to allow it in lambda expressions. For example, writing a lambda that accepts a `std::vector` containing elements of any type (but not another container) is not possible with the current syntax for generic lambdas. Instead, one must write a catch-all generic lambda that accepts any type, and then assume that it is of the proper type, or check that it is not through other means:

```

template <typename T> struct is_std_vector           : std::false_type { };
template <typename T> struct is_std_vector<std::vector<T>> : std::true_type { };

auto f = [](auto vector) {
    static_assert(is_std_vector<decltype(vector)>::value, "");
};

```

In addition to being verbose, calling the lambda with a type that is not a `std::vector` will result in a hard error inside the body of the lambda, not a template argument deduction failure. This does not play nicely with other parts of the language such as SFINAE-based detection, and it is obviously not as clear as the equivalent function template.

Another instance where "pattern matching" would be useful is to deconstruct the type of arguments that are template specializations. For example, imagine that we want to get the type of elements stored in the vector in the previous example. Right now, we'd have to write this:

```

auto f = [](auto vector) {
    using T = typename decltype(vector)::value_type;
    // ...
};

```

This is cumbersome syntax-wise, and it requires the type to provide a nested alias that does just the right thing. This is not a problem for `std::vector`, but most types don't provide such aliases (and in many cases it wouldn't make sense for them to). Hence, right now, types that do not provide nested aliases or accompanying metafunctions can simply not be deconstructed in lambdas. Instead, it would be much simpler and more flexible to write

```

auto f = []<typename T>(std::vector<T> vector) {
    // ...
};

```

2. It is often useful to retrieve the type of the parameter of a generic lambda, e.g. for accessing a static member function or an alias nested inside it. However, retrieving such a type requires using `decltype`, which includes its reference and cv qualifiers. This can often lead to unexpected results:

```

auto f = [](auto const& x) {
    using T = decltype(x);
    T copy = x;
    T::static_function();
    using Iterator = typename T::iterator;
};

```

// Compiles, but wrong semantics!
// Does not compile!
// Does not compile!

To work around this unfortunate situation, one must introduce some amount of verbosity:

```

auto f = [](auto const& x) {
    using T = std::decay_t

```

Furthermore, this problem compounds when trying to make a parameter type dependent on a previous parameter type, because aliases can't be introduced in that context to reduce verbosity:

```

auto advance = [](auto& it,
                  typename std::decay_t

```

Instead, it would be much nicer and closer to usual templates if we could simply write

```

auto f = []<typename T>(T const& x) {
    T copy = x;
    T::static_function();
    using Iterator = typename T::iterator;
};

auto advance = []<typename It>(It& it, typename It::difference_type n) {
    // ...
};

```

3. Perfect forwarding in generic lambdas is more verbose than it needs to be, and the syntax for it is different from what's usually done in normal function templates. While this is technically a direct corollary of the previous point, the author thinks this is sufficiently annoying to be

worth mentioning separately. The problem is that since the only way to get an argument’s type in a lambda is to use `decltype`, we must resort to the following syntax for perfect forwarding:

```
auto f = [](auto&& ...args) {
    return foo(std::forward<decltype(args)>(args)...);
};
```

Exactly why this works is explained in a blog post written by Scott Meyers [Meyers], but the very fact that Meyers had to write a blog post about it is telling. Indeed, the interaction between template argument deduction and reference collapsing rules is already sufficiently complicated that many C++ users would benefit from the cognitive load reduction allowed by a single perfect forwarding syntax for both lambdas and normal functions:

```
auto f = []<typename ...T>(T&& ...args) {
    return foo(std::forward<T>(args)...);
};
```

4 Proposed Wording

The wording is based on the C++17 DIS [N4659]. At the very beginning of [expr.prim.lambda] 8.1.5, change:

lambda-expression:

lambda-introducer *lambda-declarator*_{opt} *compound-statement*
~~*lambda-introducer* *<template-parameter-list>* *lambda-declarator*_{opt} *compound-statement*~~

Change in [expr.prim.lambda.closure] 8.1.5.1/3:

The closure type for a non-generic *lambda-expression* has a public inline function call operator (16.5.4) whose parameters and return type are described by the *lambda-expression*’s *parameter-declaration-clause* and *trailing-return-type* respectively. For a generic lambda, the closure type has a public inline function call operator member template (17.5.2) whose *template-parameter-list* consists of the specified *template-parameter-list*, if any, to which is appended one invented type *template-parameter* for each occurrence of **auto** in the lambda’s *parameter-declaration-clause*, in order of appearance. The invented type *template-parameter* is a parameter pack if the corresponding *parameter-declaration* declares a function parameter pack (11.3.5). The return type and function parameters of the function call operator template are derived from the *lambda-expression*’s *trailing-return-type* and *parameter-declaration-clause* by replacing each occurrence of **auto** in the *decl-specifiers* of the *parameter-declaration-clause* with the name of the corresponding invented *template-parameter*.

Change in [expr.prim.lambda.closure] 8.1.5.1/6:

[...] For a generic lambda with no *lambda-capture*, the closure type has a conversion function template to pointer to function. The conversion function template has the same invented ~~*template-parameter-list*~~ template parameter list, and the pointer to function has the same parameter types, as the function call operator template. [...]

Remove [dcl.spec.auto] 10.1.7.4/3 (to move it to the end of [expr.prim.lambda] 8.1.5):

~~If the **auto** type-specifier appears as one of the decl-specifiers in the decl-specifier-seq of a parameter-declaration of a lambda-expression, the lambda is a generic lambda (8.1.5.1).
[Example:
auto glambda = [](int i, auto a) { return i; }; // OK: a generic lambda
—end example]~~

Add at the end of [expr.prim.lambda] 8.1.5:

A lambda is a generic lambda if the **auto** type-specifier appears as one of the decl-specifiers in the decl-specifier-seq of a parameter-declaration of the lambda-expression, or if the lambda has a template-parameter-list.
[Example:
int i = [](int i, auto a) { return i; }(3, 4); // OK: a generic lambda
int j = []<class T>(T t, int i) { return i; }(3, 4); // OK: a generic lambda
— end example]

5 Implementation experience

This extension to generic lambdas had been implemented in GCC in 2009 as part of an experiment [GCC]. This extension was also part of Faisal Vali's original implementation of generic lambdas in Clang. Thus, it seems implementable.

6 Discussion

There is a question of whether to allow a lambda to contain both a *template-parameter-list* and conventional **auto**-based parameters. When allowing both syntaxes, we must also make a choice regarding the position of the invented template parameters relative to the specified template parameters.

We decided to allow mixing both syntaxes and decided to append the invented template parameters to the end of the *template-parameter-list*, because it seems like the simplest choice, it does not limit expressiveness in any way and it is consistent with what's done in the proposal for concepts [N4553].

7 Acknowledgements

Thanks to Tom Honermann, Nicol Bolas and other members of the *std-proposal* mailing list for providing comments to improve this paper.

8 References

- [N4659] Richard Smith, *Working Draft, Standard for Programming Language C++*
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4659.pdf>
- [Meyers] Scott Meyers, *C++14 Lambdas and Perfect Forwarding*
<http://scottmeyers.blogspot.com.tr/2013/05/c14-lambdas-and-perfect-forwarding.html>
- [N3418] Faisal Vali, Herb Sutter, Dave Abrahams, *Proposal for Generic (Polymorphic) Lambda Expressions*
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2012/n3418.pdf>
- [N3559] Faisal Vali, Herb Sutter, Dave Abrahams, *Proposal for Generic (Polymorphic) Lambda Expressions (Revision 2)*
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3559.pdf>
- [GCC] Adam Butcher, *Latest experimental polymorphic lambda patches*
<http://gcc.gnu.org/ml/gcc/2009-08/msg00174.html>
- [N4553] Andrew Sutton, *Working Draft, C++ extensions for Concepts*
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4553.pdf>